

Name : Venna Meenakshi Reddy

Register Number : 192471048

Brute Force - Sorting Algorithms

Selection Sort, Bubble Sort, Insertion Sort; compare execution times

Dr.Chandravathi C

Sorting Algorithms — Worked Scenario Questions

Selection Sort · Bubble Sort · Insertion Sort — 5 Solved Questions Each

1. Selection Sort

Question 1: Trophy Ranking System

A sports event organizer has recorded the scores of 50 participants but needs to display only the top 5 scores on a leaderboard screen. Since only a few top values are needed (not a full sort), use selection sort's "find maximum repeatedly" approach to extract the top 5.

Approach

Instead of sorting the entire array, repeatedly scan the remaining unprocessed portion to find the maximum, move it to the front, and shrink the unprocessed region. Stop after 5 iterations since only the top 5 scores are required. This avoids the cost of sorting all 50 values fully.

Complexity Analysis

Aspect	Complexity	Explanation
Time	$O(k \times n)$	Only k passes are made instead of n , each scanning the remaining $n-i$ elements to find the max.
Space	$O(1)$	Sorting is done in place; only a copy of the input array is used for safety.

Test Cases

```
assert top_k_scores([72,88,65,90,77,95,60,83,91,68], 5) == [95,91,90,88,83]
assert top_k_scores([5,3,1], 5) == [5,3,1]    # fewer than k items
assert top_k_scores([], 3) == []              # empty input
print('All test cases passed!')
```

Question 2: Memory-Constrained Embedded Device

A low-memory IoT sensor device collects temperature readings and must sort them with minimal write/swap operations, since each write to memory is costly on this hardware. Selection sort performs at most $n-1$ swaps regardless of input order, making it preferable to bubble or insertion sort here. Implement it with a swap counter to prove the claim.

Approach

Use the standard selection sort algorithm, but only perform a swap when the found minimum is not already in the correct position. Track the number of swaps performed and compare it against $n-1$ (the theoretical maximum) to demonstrate the low write cost.

Complexity Analysis

Aspect	Complexity	Explanation
Time	$O(n^2)$	Two nested loops compare every pair of elements regardless of input order.
Writes	$\leq n - 1$	A swap only occurs when the minimum found is not already in place, bounding total memory writes.

Test Cases

```
res, sw = selection_sort_min_writes([23.5, 19.2, 25.1, 18.8, 21.4])
assert res == sorted([23.5, 19.2, 25.1, 18.8, 21.4])
assert sw <= len(res) - 1          # write bound holds

res2, sw2 = selection_sort_min_writes([1, 2, 3, 4, 5])
assert sw2 == 0                    # already sorted -> zero swaps
print('All test cases passed!')
```

Question 3: Library Book Reordering

A librarian wants to rearrange a shelf of 20 books by ID number, but physically moving a book (swap) is time-consuming, while checking labels (comparison) is fast. Model this as an array sorting problem using selection sort and report the number of physical "moves" needed.

Approach

Represent each book as its ID in an array. Apply selection sort, incrementing a move counter only on an actual swap (a physical book relocation), while comparisons (label checks) are unrestricted since they are cheap.

Complexity Analysis

Aspect	Complexity	Explanation
Time	$O(n^2)$	Comparisons dominate the runtime; for $n = 20$ this is 190 comparisons.

Physical Moves	$\leq n - 1$	Each shelf position is corrected with at most one physical swap.
----------------	--------------	--

Test Cases

```
ordered, moves = reorder_shelf([305, 102, 250, 118, 199, 400, 101])
assert ordered == sorted([305, 102, 250, 118, 199, 400, 101])
assert moves <= len(ordered) - 1
```

```
ordered2, moves2 = reorder_shelf([100, 200, 300]) # already sorted
assert moves2 == 0
print('All test cases passed!')
```

Question 4: Contest Prize Distribution

In a coding contest, you must repeatedly select the participant with the current highest score, print their name, remove them from the list, and repeat until all participants are ranked. Implement this using selection sort logic without using a separate sorting library.

Approach

Treat this as selection sort in descending order: for each position, scan the remaining participants to find the one with the highest score, swap them into place, and print their name as the next-ranked winner.

Complexity Analysis

Aspect	Complexity	Explanation
Time	$O(n^2)$	Each of the n ranking positions requires a full scan of the remaining participants.
Space	$O(1)$ extra	Ranking is produced in place using swaps; the output list just records the order.

Test Cases

```
ranking = distribute_prizes([('Asha',88), ('Ravi',95), ('Meera',79), ('Dev',95)])
scores_only = [p[1] for p in ranking]
assert scores_only == sorted(scores_only, reverse=True) # descending order
assert ranking[0][1] == 95                             # top score first
print('All test cases passed!')
```

Question 5: Small Dataset in a Mobile App

A shopping app needs to sort a "recently viewed" list of just 8–10 items by price whenever the user opens the screen. Argue whether selection sort is a reasonable choice for such a small, frequently-refreshed list, and implement it with timing to support your answer.

Approach

For very small n , the $O(n^2)$ cost of selection sort is negligible in absolute terms (a handful of microseconds), and its simplicity and low swap count make it a reasonable, easy-to-maintain choice rather than pulling in a general-purpose sort for 8–10 items.

Complexity Analysis

Aspect	Complexity	Explanation
Time	$O(n^2)$, $n \leq 10$	For such small n , absolute execution time is in the microsecond range, so complexity class is not a practical concern.
Space	$O(1)$	In-place sort, no extra memory beyond the working copy.

Test Cases

```
assert selection_sort([499,129,899,45,275,60,310,150]) ==
sorted([499,129,899,45,275,60,310,150])
assert selection_sort([]) == []           # empty list
assert selection_sort([7]) == [7]        # single item
print('All test cases passed!')
```

2. Bubble Sort

Question 1: Exam Result Slip Correction

A teacher has a nearly sorted list of student roll numbers (only 2–3 are out of place due to late corrections). Use optimized bubble sort (with an early-exit flag) to fix the order and show it takes very few passes.

Approach

Run bubble sort with a 'swapped' flag that tracks whether any swap occurred in a pass. If a pass completes with no swaps, the list is already sorted and the algorithm exits early — ideal for nearly sorted data like this.

Complexity Analysis

Aspect	Complexity	Explanation
Best Case	$O(n)$	One pass detects a fully sorted array and exits when no swaps occur.
This Scenario	$\approx O(n)$	Only 2–3 misplaced roll numbers means very few passes are needed before the flag stays false.

Test Cases

```
sorted_rolls, passes = optimized_bubble_sort([101,102,104,103,105,107,106,108])
assert sorted_rolls == sorted([101,102,104,103,105,107,106,108])
assert passes < 8                # fewer than full n passes

sorted_ok, passes_ok = optimized_bubble_sort([1,2,3,4,5]) # already sorted
assert passes_ok == 1                # exits after first pass
print('All test cases passed!')
```

Question 2: Traffic Signal Priority Queue

At an intersection, vehicle priorities (ambulance > bus > car) arrive in a small queue and need constant reordering as new vehicles arrive. Simulate this using bubble sort each time a new vehicle is added, and discuss whether bubble sort's simplicity outweighs its inefficiency here.

Approach

Assign a numeric priority to each vehicle type. Whenever a vehicle joins the queue, append it and re-run bubble sort on the (small) queue to restore priority order. Since the queue is small and rarely has more than a handful of vehicles, bubble sort's $O(n^2)$ cost is negligible, and its simplicity makes it easy to reason about and verify in a safety-critical system.

Complexity Analysis

Aspect	Complexity	Explanation
Time	$O(n^2)$	Re-sorting the whole small queue on every arrival is acceptable since n stays very small.
Trade-off	Simplicity vs. efficiency	Bubble sort is easy to verify for correctness in a safety-critical setting, which outweighs its inefficiency at this scale.

Test Cases

```
queue = ['car', 'car', 'bus']
queue.append('ambulance')
result = bubble_sort_queue(queue)
assert result == ['ambulance', 'bus', 'car', 'car']
```

```
assert bubble_sort_queue(['ambulance']) == ['ambulance'] # single vehicle
print('All test cases passed!')
```

Question 3: Bubble Sort Visualization Tool

You are building an educational tool to visually demonstrate how "bubbles" (largest elements) rise to the top with each pass. Implement bubble sort so it prints/returns the state of the array after every single pass, useful for animation.

Approach

Modify standard bubble sort so that after each outer-loop pass, the current state of the array is stored (or printed) as a frame. These frames can then be fed into an animation or step-through UI.

Complexity Analysis

Aspect	Complexity	Explanation
Time	$O(n^2)$	Same as standard bubble sort; storing frames adds $O(n)$ work per pass.
Space	$O(n^2)$ worst case	Up to n frames are stored, each of size n , useful for driving an animation.

Test Cases

```
frames = bubble_sort_with_frames([5, 1, 4, 2, 8])
assert frames[-1] == sorted([5, 1, 4, 2, 8]) # final frame is sorted
assert frames[0] == [5, 1, 4, 2, 8]         # first frame is the original input
assert len(frames) >= 2                     # at least an initial and final frame
print('All test cases passed!')
```

Question 4: Sorting Sensor Alerts by Severity

A monitoring dashboard receives 15 alerts per minute that must be sorted by severity level (1 = critical to 5 = low) for display. Use bubble sort and measure whether the early-termination optimization meaningfully improves performance on this small, frequently-updated dataset.

Approach

Implement both a plain bubble sort and an optimized version with the early-exit flag, then compare pass counts and comparisons on the same alert data to see how much the optimization helps for this small, often-partially-sorted stream.

Complexity Analysis

Aspect	Complexity	Explanation
Plain Bubble Sort	$O(n^2)$ always	Always runs the full $n-1$ passes regardless of how sorted the data already is.
Optimized Bubble Sort	$O(n)$ best, $O(n^2)$ worst	Exits early once a pass makes no swaps, which helps when alerts arrive already close to sorted.

Test Cases

```

alerts = [2,1,3,2,1,4,3,2,5,1,2,3,4,1,2]
r1, c1 = bubble_sort_plain(alerts)
r2, c2 = bubble_sort_optimized(alerts)
assert r1 == r2 == sorted(alerts)  # both produce the same sorted result
assert c2 <= c1                    # optimized never does more comparisons
print('Plain comparisons:', c1, '| Optimized comparisons:', c2)
print('All test cases passed!')

```

Question 5: Card Game Hand Sorting

In a card game app, a player's hand of 13 cards must be sorted by rank whenever a new card is drawn. Since only one card is added at a time (nearly sorted data), implement bubble sort and compare its pass count to a full re-sort from scratch.

Approach

Use optimized bubble sort after appending the new card. Because only one card is out of place, the algorithm should need very few passes (proportional to how far the new card must travel) compared to sorting the entire hand from an unsorted state.

Complexity Analysis

Aspect	Complexity	Explanation
Nearly Sorted Hand	$\approx O(n)$	Only the newly added card needs to “bubble” into place, so few passes are needed.
Fully Shuffled Hand	$O(n^2)$	A randomly shuffled hand requires close to the maximum number of passes and swaps.

Test Cases

```

hand = [2, 4, 6, 8, 9, 11, 13]
hand.append(7)  # nearly sorted: one new card
final_hand, passes_incremental = bubble_sort_hand(hand)
assert final_hand == sorted(hand)

import random
shuffled = hand.copy()

```

```

random.shuffle(shuffled)
_, passes_full = bubble_sort_hand(shuffled)
assert passes_incremental <= passes_full # nearly-sorted needs no more passes
print('All test cases passed!')

```

3. Insertion Sort

Question 1: Live Leaderboard Updates

An online game updates a leaderboard every time a single player's score changes, while the rest of the list remains sorted. Use insertion sort to insert just the updated score into its correct position instead of re-sorting the entire list, and measure the efficiency gain.

Approach

Remove the updated player's old entry, then use the insertion-sort technique of shifting elements to find and insert the new score into its correct position in the already-sorted leaderboard, avoiding a full re-sort.

Complexity Analysis

Aspect	Complexity	Explanation
Time	$O(n)$	Only the shift needed to place the single new score is performed, instead of an $O(n \log n)$ or $O(n^2)$ full re-sort.
Space	$O(1)$ extra	Insertion happens via in-place shifting.

Test Cases

```

board = [980, 875, 760, 690, 500]
updated_board, shifts = insert_updated_score(board, 820)
assert updated_board == [980, 875, 820, 760, 690, 500]

board2, shifts2 = insert_updated_score(board, 100) # lowest score
assert board2[-1] == 100 and shifts2 == 0          # no shifts needed, goes to the end
print('All test cases passed!')

```

Question 2: Card Sorting While Playing

A card player sorts their hand as they pick up cards one at a time (the classic real-world analogy for insertion sort). Implement this behavior programmatically, sorting a growing hand card by card.

Approach

Maintain a hand list. Each time a new card is picked up, insert it into its correct position among the already-sorted cards by shifting larger cards one position to the right, exactly like insertion sort's inner loop.

Complexity Analysis

Aspect	Complexity	Explanation
Time	$O(n)$ per card, $O(n^2)$ total	Each new card may require shifting past several existing cards; over n cards this totals $O(n^2)$.
Space	$O(1)$ extra	Cards are shifted in place within the hand list.

Test Cases

```
hand = []
for card in [7, 2, 9, 4, 1]:
    hand = pick_up_card(hand, card)
assert hand == sorted([7, 2, 9, 4, 1])

assert pick_up_card([], 5) == [5]    # first card picked up
print('All test cases passed!')
```

Question 3: Real-Time Stock Price Feed

A trading dashboard receives stock prices one at a time and must maintain a sorted list at all times for quick min/max lookups. Use insertion sort to insert each new incoming price efficiently into the already-sorted list.

Approach

Keep a running sorted list of prices. For each incoming price, use binary search (or a linear scan) to find its correct position, then shift subsequent elements right by one and insert it — the core idea of insertion sort applied incrementally.

Complexity Analysis

Aspect	Complexity	Explanation
Time (binary search variant)	$O(\log n)$ search + $O(n)$ shift	Finding the position is fast, but shifting array elements to make room is still $O(n)$ per insertion.
Overall for n prices	$O(n^2)$ worst case	Still quadratic overall due to shifting, but far better than re-sorting the whole list from scratch each time.

Test Cases

```
prices = []
for p in [102.5, 98.3, 105.1, 100.0, 97.8]:
    prices = insert_price(prices, p)
assert prices == sorted([102.5, 98.3, 105.1, 100.0, 97.8])
assert prices[0] == min(prices) and prices[-1] == max(prices) # O(1) min/max lookup
print('All test cases passed!')
```

Question 4: Playlist Reordering by Recently Added

A music app lets users insert a newly added song into a playlist that is already sorted by duration. Implement insertion sort logic to place just the new song correctly instead of sorting the whole playlist again.

Approach

Append the new song's duration to the playlist and shift it leftward past any longer songs until it reaches its correct sorted position, exactly as the inner loop of insertion sort does for a single new element.

Complexity Analysis

Aspect	Complexity	Explanation
Time	$O(n)$	Only the new song is shifted into place among already-sorted songs.
Space	$O(1)$ extra	Insertion is performed via in-place shifting within the list.

Test Cases

```
playlist = [('Intro', 120), ('Chill Beat', 210), ('Long Jam', 340)]
updated_playlist = insert_song(playlist, ('Quick Track', 180))
durations = [s[1] for s in updated_playlist]
assert durations == sorted([120, 210, 340, 180])
assert ('Quick Track', 180) in updated_playlist
print('All test cases passed!')
```

Question 5: Nearly Sorted Sensor Log Correction

A weather station logs temperature readings in mostly chronological order, but occasional delayed transmissions cause a few entries to be out of place. Use insertion sort to fix the log

and show that the number of shifts is very low compared to a randomly shuffled version of the same data.

Approach

Run standard insertion sort on the log and count total shifts performed. Because only a few entries are out of place, most elements require zero or one shift, resulting in a shift count close to the best-case $O(n)$ rather than the worst-case $O(n^2)$.

Complexity Analysis

Aspect	Complexity	Explanation
Nearly Sorted Data	$\approx O(n)$	Very few shifts are needed since most elements are already close to their correct position.
Randomly Shuffled Data	$O(n^2)$	Requires close to the maximum number of shifts, illustrating insertion sort's sensitivity to input order.

Test Cases

```
log = [18.2, 18.5, 18.9, 17.9, 19.1, 19.4, 19.0]
sorted_log, shifts_nearly = insertion_sort_count_shifts(log)
assert sorted_log == sorted(log)

import random
shuffled_log = log.copy()
random.shuffle(shuffled_log)
_, shifts_random = insertion_sort_count_shifts(shuffled_log)
print('Nearly-sorted shifts:', shifts_nearly, '| Random shifts:', shifts_random)
print('All test cases passed!')
```

1. Selection Sort

Question 1: Trophy Ranking System

Programiz

Python Online Compiler

main.py

1- def top_k_scores(scores, k):

2 arr = scores.copy() # Copy the original list

3 n = len(arr)

4

5 # If array is empty

6- if n == 0:

7 return []

8

9 # If k is greater than the number of elements

10 k = min(k, n)

11

12 # Find the maximum repeatedly (Selection Sort approach)

13- for i in range(k):

14 max_index = i

15

16- for j in range(i + 1, n):

17- if arr[j] > arr[max_index]:

18 max_index = j

19

20 # Swap maximum element to the current position

21 arr[i], arr[max_index] = arr[max_index], arr[i]

22

23 # Return only the top k scores

24 return arr[:k]

25

26

27 # Test Cases

28 assert top_k_scores([72,88,65,90,77,95,60,83,91,68], 5) == [95,91,90,88,83]

29 assert top_k_scores([5,3,1], 5) == [5,3,1] # Fewer than k items

30 assert top_k_scores([], 3) == [] # Empty input

31

32 print("All test cases passed!")

Output

All test cases passed!

=== Code Execution Successful ===

Question 2: Memory-Constrained Embedded Device

Programiz

Python Online Compiler

main.py

1- def selection_sort_min_writes(arr):

2 arr = arr.copy() # Preserve original list

3 n = len(arr)

4 swaps = 0

5

6- for i in range(n - 1):

7 min_index = i

8

9 # Find the minimum element

10- for j in range(i + 1, n):

11- if arr[j] < arr[min_index]:

12 min_index = j

13

14 # Swap only if needed

15- if min_index != i:

16 arr[i], arr[min_index] = arr[min_index], arr[i]

17 swaps += 1

18

19 return arr, swaps

20

21

22 # Test Cases

23 res, sw = selection_sort_min_writes([23.5, 19.2, 25.1, 18.8, 21.4])

24 assert res == sorted([23.5, 19.2, 25.1, 18.8, 21.4])

25 assert sw <= len(res) - 1 # Write bound holds

26

27 res2, sw2 = selection_sort_min_writes([1, 2, 3, 4, 5])

28 assert res2 == [1, 2, 3, 4, 5]

29 assert sw2 == 0 # Already sorted → zero swaps

30

31 print("All test cases passed!")

Output

All test cases passed!

=== Code Execution Successful ===

Question 3: Library Book Reordering

Programiz
Python Online Compiler

```
main.py
1 def reorder_shelf(books):
2     books = books.copy() # Keep original list unchanged
3     n = len(books)
4     moves = 0
5
6     # Selection Sort
7     for i in range(n - 1):
8         min_index = i
9
10        # Find the smallest book ID
11        for j in range(i + 1, n):
12            if books[j] < books[min_index]:
13                min_index = j
14
15        # Swap only if necessary
16        if min_index != i:
17            books[i], books[min_index] = books[min_index], books[i]
18            moves += 1
19
20    return books, moves
21
22
23 # Test Cases
24 ordered, moves = reorder_shelf([305, 102, 250, 118, 199, 400, 101])
25 assert ordered == sorted([305, 102, 250, 118, 199, 400, 101])
26 assert moves <= len(ordered) - 1
27
28 ordered2, moves2 = reorder_shelf([100, 200, 300]) # Already sorted
29 assert ordered2 == [100, 200, 300]
30 assert moves2 == 0
31
32 print("All test cases passed!")
```

Output

All test cases passed!

=== Code Execution Successful ===

Question 4: Contest Prize Distribution

Programiz
Python Online Compiler

```
main.py
1 def distribute_prizes(participants):
2     participants = participants.copy() # Preserve original list
3     n = len(participants)
4
5     # Selection Sort in Descending Order
6     for i in range(n - 1):
7         max_index = i
8
9         # Find participant with highest score
10        for j in range(i + 1, n):
11            if participants[j][1] > participants[max_index][1]:
12                max_index = j
13
14        # Swap if necessary
15        if max_index != i:
16            participants[i], participants[max_index] = participants[max_index], participants[i]
17
18        # Print ranking
19        print("Prize Distribution Ranking:")
20        for rank, (name, score) in enumerate(participants, start=1):
21            print(f"{rank}. {name} - {score}")
22
23    return participants
24
25
26 # Test Cases
27 ranking = distribute_prizes([('Asha', 88), ('Ravi', 95), ('Meera', 79), ('Dev', 95)])
28
29 scores_only = [p[1] for p in ranking]
30 assert scores_only == sorted(scores_only, reverse=True) # Descending order
31 assert ranking[0][1] == 95 # Top score first
32
33 print("All test cases passed!")
```

Output

Prize Distribution Ranking:

1. Ravi - 95

2. Dev - 95

3. Asha - 88

4. Meera - 79

All test cases passed!

=== Code Execution Successful ===

Question 5: Small Dataset in a Mobile App

Programiz
Python Online Compiler

```
main.py
1 def optimized_bubble_sort(arr):
2     arr = arr.copy() # Preserve original list
3     n = len(arr)
4     passes = 0
5
6     for i in range(n - 1):
7         swapped = False
8         passes += 1
9
10        # Bubble the largest element to the end
11        for j in range(n - 1 - i):
12            if arr[j] > arr[j + 1]:
13                arr[j], arr[j + 1] = arr[j + 1], arr[j]
14                swapped = True
15
16        # Exit early if no swaps occurred
17        if not swapped:
18            break
19
20    return arr, passes
21
22
23 # Example Input
24 roll_numbers = [101, 102, 104, 103, 105, 107, 106, 108]
25
26 sorted_rolls, passes = optimized_bubble_sort(roll_numbers)
27
28 print("Original Roll Numbers :", roll_numbers)
29 print("Sorted Roll Numbers :", sorted_rolls)
30 print("Number of Passes :", passes)
31
32 # Test Cases
33 sorted_rolls, passes = optimized_bubble_sort([101, 102, 104, 103, 105, 107, 106, 108])
34 assert sorted_rolls == sorted([101, 102, 104, 103, 105, 107, 106, 108])
35 assert passes < 8 # Fewer than full n passes
36
37 sorted_ok, passes_ok = optimized_bubble_sort([1, 2, 3, 4, 5]) # Already sorted
38 assert passes_ok == 1 # Exits after first pass
39
40 print("All test cases passed!")
```

Output

```
Original Prices : [499, 129, 899, 45, 275, 60, 310, 150]
Sorted Prices   : [45, 60, 129, 150, 275, 310, 499, 899]
Execution Time  : 1.4659948647022247e-05 seconds
All test cases passed!

=== Code Execution Successful ===
```

2. Bubble Sort

Question 1: Exam Result Slip Correction

Programiz
Python Online Compiler

```
main.py
1 def optimized_bubble_sort(arr):
2     arr = arr.copy() # Preserve original list
3     n = len(arr)
4     passes = 0
5
6     for i in range(n - 1):
7         swapped = False
8         passes += 1
9
10        # Bubble the largest element to the end
11        for j in range(n - 1 - i):
12            if arr[j] > arr[j + 1]:
13                arr[j], arr[j + 1] = arr[j + 1], arr[j]
14                swapped = True
15
16        # Exit early if no swaps occurred
17        if not swapped:
18            break
19
20    return arr, passes
21
22
23 # Example Input
24 roll_numbers = [101, 102, 104, 103, 105, 107, 106, 108]
25
26 sorted_rolls, passes = optimized_bubble_sort(roll_numbers)
27
28 print("Original Roll Numbers :", roll_numbers)
29 print("Sorted Roll Numbers :", sorted_rolls)
30 print("Number of Passes :", passes)
31
32 # Test Cases
33 sorted_rolls, passes = optimized_bubble_sort([101, 102, 104, 103, 105, 107, 106, 108])
34 assert sorted_rolls == sorted([101, 102, 104, 103, 105, 107, 106, 108])
35 assert passes < 8 # Fewer than full n passes
36
37 sorted_ok, passes_ok = optimized_bubble_sort([1, 2, 3, 4, 5]) # Already sorted
38 assert passes_ok == 1 # Exits after first pass
39
40 print("All test cases passed!")
```

Output

```
Original Roll Numbers : [101, 102, 104, 103, 105, 107, 106, 108]
Sorted Roll Numbers   : [101, 102, 103, 104, 105, 106, 107, 108]
Number of Passes      : 2
All test cases passed!

=== Code Execution Successful ===
```

Question 2: Traffic Signal Priority Queue

Programiz
Python Online Compiler

```
main.py
1- def bubble_sort_queue(queue):
2     queue = queue.copy() # Preserve original queue
3
4     # Priority values (higher number = higher priority)
5     priority = {
6         "ambulance": 3,
7         "bus": 2,
8         "car": 1
9     }
10
11     n = len(queue)
12
13     # Bubble Sort (Descending by Priority)
14     for i in range(n - 1):
15         swapped = False
16
17         for j in range(n - 1 - i):
18             if priority[queue[j]] < priority[queue[j + 1]]:
19                 queue[j], queue[j + 1] = queue[j + 1], queue[j]
20                 swapped = True
21
22         if not swapped:
23             break
24
25     return queue
26
27
28 # Example Input
29 queue = ["car", "car", "bus"]
30
31 # New vehicle arrives
32 queue.append("ambulance")
33
34 print("Original Queue :", queue)
35
36 result = bubble_sort_queue(queue)
37
38 print("Priority Queue :", result)
39
40 # Test Case
41 assert result == ["ambulance", "bus", "car", "car"]
42
43 print("All test cases passed!")
```

Output

```
Original Queue : ['car', 'car', 'bus', 'ambulance']
Priority Queue : ['ambulance', 'bus', 'car', 'car']
All test cases passed!

=== Code Execution Successful ===
```

Question 3: Bubble Sort Visualization Tool

Programiz
Python Online Compiler

```
main.py
1- def bubble_sort_with_frames(arr):
2     arr = arr.copy() # Preserve original list
3     n = len(arr)
4
5     # Store the initial array as the first frame
6     frames = [arr.copy()]
7
8     # Bubble Sort
9     for i in range(n - 1):
10         swapped = False
11
12         for j in range(n - 1 - i):
13             if arr[j] > arr[j + 1]:
14                 arr[j], arr[j + 1] = arr[j + 1], arr[j]
15                 swapped = True
16
17         # Store the array after each pass
18         frames.append(arr.copy())
19
20         # Exit early if already sorted
21         if not swapped:
22             break
23
24     return frames
25
26
27 # Example Input
28 frames = bubble_sort_with_frames([5, 1, 4, 2, 8])
29
30 print("Bubble Sort Frames:")
31 for i, frame in enumerate(frames):
32     print(f"Pass {i}: {frame}")
33
34 # Test Cases
35 assert frames[-1] == sorted([5, 1, 4, 2, 8]) # Final frame is sorted
36 assert frames[0] == [5, 1, 4, 2, 8] # First frame is original input
37 assert len(frames) >= 2 # At least initial and final frame
38
39 print("All test cases passed!")
```

Output

```
Bubble Sort Frames:
Pass 0: [5, 1, 4, 2, 8]
Pass 1: [1, 4, 2, 5, 8]
Pass 2: [1, 2, 4, 5, 8]
Pass 3: [1, 2, 4, 5, 8]
All test cases passed!

=== Code Execution Successful ===
```

Question 4: Sorting Sensor Alerts by Severity

```
Programiz
Python Online Compiler

main.py
1- def bubble_sort_plain(arr):
2     arr = arr.copy()
3     n = len(arr)
4     comparisons = 0
5
6     for i in range(n - 1):
7         for j in range(n - 1 - i):
8             comparisons += 1
9             if arr[j] > arr[j + 1]:
10                 arr[j], arr[j + 1] = arr[j + 1], arr[j]
11
12     return arr, comparisons
13
14
15- def bubble_sort_optimized(arr):
16     arr = arr.copy()
17     n = len(arr)
18     comparisons = 0
19
20     for i in range(n - 1):
21         swapped = False
22
23         for j in range(n - 1 - i):
24             comparisons += 1
25             if arr[j] > arr[j + 1]:
26                 arr[j], arr[j + 1] = arr[j + 1], arr[j]
27                 swapped = True
28
29         if not swapped:
30             break
31
32     return arr, comparisons
33
34
35 # Example Input
36 alerts = [2,1,3,2,1,4,3,2,5,1,2,3,4,1,2]
37
38 r1, c1 = bubble_sort_plain(alerts)
39 r2, c2 = bubble_sort_optimized(alerts)
40
41 print("Original Alerts      :", alerts)
42 print("Sorted Alerts (Plain)  :", r1)
43 print("Sorted Alerts (Optimized) :", r2)
44 print("Plain Comparisons      :", c1)
45 print("Optimized Comparisons   :", c2)
46
47 # Test Cases
48 assert r1 == r2 == sorted(alerts) # Both produce the same sorted result
49 assert c2 <= c1 # Optimized never does more comparisons
50
51 print("Plain comparisons:", c1, ", ", "Optimized comparisons:", c2)
52 print("All test cases passed!")

Output
Original Alerts      : [2, 1, 3, 2, 1, 4, 3, 2, 5, 1, 2, 3, 4, 1, 2]
Sorted Alerts (Plain)  : [1, 1, 1, 1, 2, 2, 2, 2, 2, 3, 3, 3, 3, 4, 4, 5]
Sorted Alerts (Optimized) : [1, 1, 1, 1, 2, 2, 2, 2, 2, 3, 3, 3, 3, 4, 4, 5]
Plain Comparisons      : 105
Optimized Comparisons   : 89
Plain comparisons: 105 | Optimized comparisons: 89
All test cases passed!

=== Code Execution Successful ===
```

Question 5: Card Game Hand Sorting

```
Programiz
Python Online Compiler

main.py
1 import random
2
3- def bubble_sort_hand(arr):
4     arr = arr.copy()
5     n = len(arr)
6     passes = 0
7
8     for i in range(n - 1):
9         swapped = False
10         passes += 1
11
12         for j in range(n - 1 - i):
13             if arr[j] > arr[j + 1]:
14                 arr[j], arr[j + 1] = arr[j + 1], arr[j]
15                 swapped = True
16
17         # Stop if already sorted
18         if not swapped:
19             break
20
21     return arr, passes
22
23
24 # Test Cases
25
26 hand = [2, 4, 6, 8, 9, 11, 13] # Nearly sorted: one new card
27 hand.append(7)
28
29 final_hand, passes_incremental = bubble_sort_hand(hand)
30 assert final_hand == sorted(hand)
31
32 shuffled = hand.copy()
33 random.shuffle(shuffled)
34
35 ..., passes_full = bubble_sort_hand(shuffled)
36
37 assert passes_incremental <= passes_full # Nearly sorted needs no more passes
38
39 print("All test cases passed!")

Output
All test cases passed!

=== Code Execution Successful ===
```


3. Insertion Sort

Question 1: Live Leaderboard Updates

Programiz
Python Online Compiler

main.py

```
1- def insert_updated_score(board, new_score):
2-     board = board.copy() # Preserve original leaderboard
3-     shifts = 0
4-
5-     # Add the new score at the end
6-     board.append(new_score)
7-
8-     i = len(board) - 2
9-
10-    # Shift smaller scores to the right (descending order)
11-    while i >= 0 and board[i] < new_score:
12-        board[i + 1] = board[i]
13-        shifts += 1
14-        i -= 1
15-
16-    # Insert the new score
17-    board[i + 1] = new_score
18-
19-    return board, shifts
20-
21-
22- # Test Cases
23- board = [980, 875, 760, 690, 500]
24-
25- updated_board, shifts = insert_updated_score(board, 820)
26- assert updated_board == [980, 875, 820, 760, 690, 500]
27-
28- board2, shifts2 = insert_updated_score(board, 100) # Lowest score
29- assert board2[-1] == 100 and shifts2 == 0 # No shifts needed
30-
31- print("All test cases passed!")
```

Output

All test cases passed!
=== Code Execution Successful ===

Question 2: Card Sorting While Playing

Programiz
Python Online Compiler

main.py

```
1- def pick_up_card(hand, card):
2-     hand = hand.copy() # Preserve original hand
3-
4-     # Add the new card
5-     hand.append(card)
6-
7-     # Insert the card into its correct position
8-     i = len(hand) - 2
9-     while i >= 0 and hand[i] > card:
10-         hand[i + 1] = hand[i]
11-         i -= 1
12-
13-     hand[i + 1] = card
14-
15-     return hand
16-
17-
18- # Test Cases
19- hand = []
20-
21- for card in [7, 2, 9, 4, 1]:
22-     hand = pick_up_card(hand, card)
23-
24- assert hand == sorted([7, 2, 9, 4, 1])
25-
26- assert pick_up_card([], 5) == [5] # First card picked up
27-
28- print("All test cases passed!")
```

Output

All test cases passed!
=== Code Execution Successful ===

Question 3: Real-Time Stock Price Feed

Programiz
Python Online Compiler

```
main.py
1- def insert_price(prices, new_price):
2-     prices = prices.copy()    # Preserve original list
3-
4-     # Add the new price at the end
5-     prices.append(new_price)
6-
7-     # Shift larger elements to the right
8-     i = len(prices) - 2
9-     while i >= 0 and prices[i] > new_price:
10-         prices[i + 1] = prices[i]
11-         i -= 1
12-
13-     # Insert the new price
14-     prices[i + 1] = new_price
15-
16-     return prices
17-
18- # Test Cases
19- prices = []
20-
21-
22- for p in [102.5, 98.3, 105.1, 100.0, 97.8]:
23-     prices = insert_price(prices, p)
24-
25- assert prices == sorted([102.5, 98.3, 105.1, 100.0, 97.8])
26- assert prices[0] == min(prices) and prices[-1] == max(prices)    # O(1) min/max lookup
27-
28- print("All test cases passed!")
```

Output

All test cases passed!

=== Code Execution Successful ===

Question 4: Playlist Reordering by Recently Added

Programiz
Python Online Compiler

```
main.py
1- def insert_song(playlist, new_song):
2-     playlist = playlist.copy()    # Preserve original playlist
3-
4-     # Add the new song at the end
5-     playlist.append(new_song)
6-
7-     # Insert the new song based on duration (ascending order)
8-     i = len(playlist) - 2
9-
10-     while i >= 0 and playlist[i][1] > new_song[1]:
11-         playlist[i + 1] = playlist[i]
12-         i -= 1
13-
14-     playlist[i + 1] = new_song
15-
16-     return playlist
17-
18- # Test Cases
19- playlist = [("Intro", 120), ("Chill Beat", 210), ("Long Jam", 340)]
20-
21-
22- updated_playlist = insert_song(playlist, ("Quick Track", 180))
23-
24- durations = [s[1] for s in updated_playlist]
25-
26- assert durations == sorted([120, 210, 340, 180])
27- assert ("Quick Track", 180) in updated_playlist
28-
29- print("All test cases passed!")
```

Output

All test cases passed!

=== Code Execution Successful ===

Question 5: Nearly Sorted Sensor Log Correction



Python Online Compiler

```
1 import random
2
3 def insertion_sort_count_shifts(arr):
4     arr = arr.copy() # Preserve original list
5     shifts = 0
6
7     for i in range(1, len(arr)):
8         key = arr[i]
9         j = i - 1
10
11         # Shift elements greater than key
12         while j >= 0 and arr[j] > key:
13             arr[j + 1] = arr[j]
14             shifts += 1
15             j -= 1
16
17         arr[j + 1] = key
18
19     return arr, shifts
20
21
22 # Test Cases
23 log = [18.2, 18.5, 18.9, 17.9, 19.1, 19.4, 19.0]
24
25 sorted_log, shifts_nearly = insertion_sort_count_shifts(log)
26 assert sorted_log == sorted(log)
27
28 shuffled_log = log.copy()
29 random.shuffle(shuffled_log)
30
31 _, shifts_random = insertion_sort_count_shifts(shuffled_log)
32
33 print("Nearly-sorted shifts:", shifts_nearly,
34       "| Random shifts:", shifts_random)
35
36 print("All test cases passed!")
```

Run

Share

Output

All test cases passed!

=== Code Execution Successful ===